

4.2.10 Lesson Review

Date: 3/20/2026, 2:19:20 PM

Time Spent: 12:16

Score: 100%

Passing Score: 80%



Why does limiting sensitive information in training datasets help reduce the impact of model inversion and model theft attacks?

- ☒ It minimizes the amount of personal or confidential data that could be exposed even if an attacker reconstructs or clones the model. ✓ Correct
- ☐ It forces the model to memorize all remaining records, making them harder for attackers to decode from outputs.
- ☐ It prevents attackers from sending high-volume queries to the model, since the system will automatically reject requests that appear to target sensitive content and detailed attributes across multiple users.
- ☐ It guarantees that attackers cannot infer any attributes about users because the model no longer relies on statistical patterns.

Explanation

Minimizing the amount of personal or confidential data that could be exposed even if an attacker reconstructs or clones the model is the correct answer. Reducing sensitive content in the training data limits what can be exposed if an attacker inverts or clones the model. Even when some reconstruction occurs, there is less personal or confidential information available for the attacker to obtain.

Forcing It forces the model to memorize all remaining records, making them harder for attackers to decode from outputs is incorrect. Limiting sensitive data does not cause the model to "hide" remaining records through memorization. In fact, memorization can worsen privacy risk by making it easier to extract specific examples or details from the model.

Guaranteeing that attackers cannot infer any attributes about users because the model no longer relies on statistical patterns is incorrect. Removing some sensitive fields does not stop the model from learning statistical relationships. Attackers may still infer private attributes indirectly from remaining patterns, so this approach cannot guarantee complete protection.

Preventing attackers from sending high-volume queries to the model is incorrect. Data minimization does not control query volume or automatically block certain requests. Throttling and input filtering are separate access controls, not direct results of limiting sensitive information in training.

Related Content

Question 2

✓ Correct

A fraud-detection model performs well on standard evaluation tests, yet in production it increasingly misclassifies a narrow set of attacker-crafted inputs as legitimate. Monitoring shows that recent production data no longer matches the patterns seen in the training dataset.

Which issue does this scenario illustrate?

- ☐ Output integrity attack
- ☐ Bias
- ☒ Model skewing ✓ Correct
- ☐ Model denial of service

Explanation

Model skewing is the correct answer. Model skewing occurs when the data a model sees in production diverges from its training data, causing systematic errors on certain inputs. Here, attacker-crafted inputs exploit that shift so the model misclassifies specific fraudulent cases while still appearing accurate overall.

Output integrity attack is incorrect. An output integrity attack focuses on how model outputs are consumed or executed by downstream systems, not on shifts in input data distributions.

Bias is incorrect. Bias involves unfair or unequal performance across demographic or protected groups, not attacker-driven distribution shifts in transaction data.

Model denial of service is incorrect. Model denial of service aims to slow down or disrupt access to the model; in this case, the model remains available but makes targeted mistakes.

Related Content

4.2.6 AI Attacks Analysis and Controls

resources\questions\q_ai_attacks_analysis_and_controls_04.question.xml

Which practice BEST demonstrates using fingerprinting or watermarking to help detect model theft?

- ☐ Requiring multifactor authentication for developers before they can view configuration settings or retrain the model on updated datasets.
- ☐ Rotating API keys frequently and enforcing per-key rate limits so each client's usage of the model can be throttled and monitored separately.
- ☒ Embedding a subtle statistical pattern in generated text so later analysis can show that a suspected clone model was trained on those outputs. ✓ Correct
- ☐ Storing all model checkpoints in a restricted network segment that uses hardware-backed encryption, intrusion detection, and strictly segmented access controls for all engineering teams.

Explanation

Embedding a subtle statistical pattern in generated text so later analysis can show that a suspected clone model was trained on those outputs is the correct answer. A hidden statistical pattern in outputs acts as a watermark that can later be tested for. If a suspected clone shows the same pattern, it indicates the clone was likely trained on those outputs.

Requiring multifactor authentication for developers before they can view configuration settings is incorrect. Multifactor authentication protects administrative access but does not place any traceable mark in the model's responses, so it cannot reveal whether outputs were used to train a copy.

Rotating API keys frequently and enforcing per-key rate limits is incorrect. API key rotation and rate limits help control and monitor usage, but they do not alter outputs or embed any signature that would identify content reused in a cloned model.

Storing all model checkpoints in a restricted network segment is incorrect. Securing checkpoints with strong storage controls reduces direct artifact theft but does not watermark outputs. It protects where the model is stored, not whether its responses were later used to build a replica.

Related Content



4.2.4 Model Inversion and Model Theft

Question 4

✓ Correct

A security team wants to detect if training files for a model are modified in storage before each retraining cycle. They implement cryptographic checksums and digital signatures to verify that files match approved versions.

Which type of control are they primarily using?

☒ Data integrity controls ✓ Correct

☐ Model guardrails

☐ Access controls

☐ Output validation

Explanation

Data integrity controls is the correct answer. Using checksums and digital signatures to confirm that training files still match approved versions is a direct use of data integrity controls, which detect unauthorized or accidental modifications.

Access controls is incorrect. These determine who can read or modify the training files through authentication and authorization, but they do not themselves check whether the files have already been altered.

Output validation is incorrect. This focuses on checking model responses or structured outputs for correctness or policy compliance, not on verifying that stored training data remains unchanged.

Model guardrails is incorrect. These constrain model behavior and outputs during inference according to safety rules; they do not verify the integrity of training files in storage.

Related Content



4.2.7 Applying Compensating Controls

resources\questions\q_applying_compensating_controls_05.question.xml

A public-facing chatbot backed by a large language model suddenly becomes unavailable because an attacker floods it with long, complex prompts that consume excessive computation and exceed token limits.

What type of issue does this scenario illustrate?

- ☐ Membership inference
- ☐ Model skewing
- ☐ Output integrity attack
- ☒ Model denial of service ✓ Correct

Explanation

Model denial of service is the correct answer. Model denial of service occurs when an attacker exhausts the resources a model depends on so it becomes slow or unavailable. Here, large volumes of long prompts consume computation and token budgets, preventing legitimate users from accessing the chatbot, which is a classic denial-of-service pattern focused on the model.

Output integrity attack is incorrect. An output integrity attack is about downstream systems treating model responses as trusted commands or data without validation; the main problem in this scenario is loss of availability, not misuse of outputs.

Membership inference is incorrect. Membership inference aims to determine whether specific records were in the training data by analyzing model outputs, which is unrelated to overwhelming the model with heavy traffic.

Model skewing is incorrect. Model skewing involves shifts between training and production data distributions that cause systematic errors, not traffic-based resource exhaustion that makes the service unavailable.

Related Content



4.2.6 AI Attacks Analysis and Controls

resources\questions\q_ai_attacks_analysis_and_controls_07.question.question.xml

A security team is reviewing an AI model used in a factory robot vision system. During testing, they notice that when a small yellow sticker appears in the corner of an object, the system consistently misclassified that object as "safe to handle" instead of "hazardous," even though all other classifications seem accurate. Which type of attack most likely explains this behavior?

- ☐ Model evasion attack using adversarial input samples at inference
- ☐ Trojanized deployment pipeline inserting a malicious model binary
- ☐ Prompt injection attack modifying the model's instructions at runtime

☒ Hidden trigger backdoor attack embedded during training

✔ Correct

Explanation

Hidden trigger backdoor attack embedded during training is correct. A hidden trigger backdoor attack is created by poisoning training data so that a specific visual pattern (such as a small sticker) causes systematic misclassification, while the model behaves normally otherwise.

Prompt injection attack modifying the model's instructions at runtime is incorrect. A prompt injection attack manipulates text-based instructions or prompts at runtime, not visual triggers embedded during training.

Model evasion attack using adversarial input samples at inference is incorrect. A model evasion attack typically relies on subtle perturbations at inference time, not on a fixed, trained-in trigger pattern.

Trojanized deployment pipeline inserting a malicious model binary is incorrect. A Trojanized deployment pipeline would focus on inserting malicious code or binaries, not on training the model to misclassify inputs only when a visual trigger appears.

Related Content



4.2.2 Backdoor and Trojan Attacks

resources\questions\q_backdoor_and_trojan_attacks_01.question.xml

Which control MOST directly reduces the risk that input manipulation attacks will cause harmful automated actions?

- ☐ Granting the model full write access to production systems so it can immediately remediate any issues it detects without delay.
- ☒ Allowing the model to propose actions on incoming data while requiring human review and approval before those actions are executed. ✓ Correct
- ☐ Ignoring all anomalies in input data distributions so the model can adapt freely without human oversight or intermediate checks.
- ☐ Automatically executing any action the model suggests as long as its confidence score exceeds a predefined numerical threshold.

Explanation

Allowing the model to propose actions on incoming data while requiring human review and approval before those actions are executed is the correct answer. Having the model suggest actions but requiring a human to approve them is a human-in-the-loop safeguard. If adversarial inputs cause bad recommendations, the human review step helps block harmful actions.

Automatically executing any action the model suggests is incorrect. Confidence scores can be misleading in adversarial cases, because manipulated inputs can make the model confidently wrong. Automatically executing based on a threshold leaves the system vulnerable to such attacks.

Ignoring all anomalies in input data distributions is incorrect. Anomalies in input data can signal adversarial manipulation. Ignoring them removes an important detection mechanism and lets manipulated inputs pass unchecked.

Granting the model full write access to production systems is incorrect. Giving broad write access magnifies the damage from any successful input manipulation. If the model is tricked, it can immediately apply harmful changes to production systems.

Related Content

 4.2.1 Analyzing AI Attacks

Question 8

✓ Correct

Which situation BEST illustrates the use of rate limiting as a compensating control for an AI-powered API?

- ☐ Forcing all user inputs to conform to a predefined prompt template.
- ☒ Restricting the number of requests a client can send to the model within a defined time period. ✓ Correct
- ☐ Requiring multi-factor authentication for every user who accesses the AI dashboard.
- ☐ Encrypting all traffic between the client and the model using TLS certificates.

Explanation

Restricting the number of requests a client can send to the model within a defined time period is the correct answer. Rate limiting specifically caps how many requests a client can make in a given time window, helping prevent abuse such as excessive querying or model denial of service.

Encrypting all traffic between the client and the model using TLS certificates is incorrect. Encryption protects data in transit but does not restrict how many requests a client can send.

Requiring multi-factor authentication for every user who accesses the AI dashboard is incorrect. Multi-factor authentication controls who gets access, not how frequently an authenticated user can query the model.

Forcing all user inputs to conform to a predefined prompt template is incorrect. Prompt templates shape and standardize input content but do not control the volume of requests being submitted.

Related Content



4.2.7 Applying Compensating Controls

resources\questions\q_applying_compensating_controls_07.question.xml

An autonomous vehicle occasionally fails to stop at certain signs during road tests. Investigators find that every ignored stop sign has a small blue sticker in the lower-right corner, while all other stop signs are handled correctly. What is MOST likely the goal of the attacker who manipulated this AI system during training?

- ☒ Cause misclassification only when a specific trigger pattern is present ✓ Correct
- ☐ Exfiltrate all road images to an external attacker-controlled server
- ☐ Inject malicious prompts into the vehicle's natural language interface
- ☐ Disable the entire model so the vehicle cannot process traffic signs

Explanation

Cause misclassification only when a specific trigger pattern is present is correct. The described behavior matches a backdoor attack, where the model is trained to misclassify only when a specific trigger (the blue sticker) appears, while functioning normally otherwise.

Exfiltrate all road images to an external attacker-controlled server is incorrect. Exfiltrating all images describes a Trojan that sends data to an attacker, which is not supported by the observed selective misclassification.

Inject malicious prompts into the vehicle's natural language interface is incorrect. There is no mention of prompt-based interaction or language interfaces; the issue is purely with visual sign recognition.

Disable the entire model so the vehicle cannot process traffic signs is incorrect. The model is not disabled; it correctly processes all other traffic signs, which contradicts the idea of a complete failure.

Related Content



4.2.2 Backdoor and Trojan Attacks

resources\questions\q_backdoor_and_trojan_attacks_02.question.xml

Which concept is most relevant when protecting the integrity of an AI model's intermediate reasoning steps so they cannot be tampered with or used to shift its behavior over time?

☒ Chain of thought ✓ Correct

☐ Output formatting

☐ Guardrail policies

☐ Rate limiting

Explanation

Chain of thought is the correct answer. Chain of thought is the model's intermediate reasoning, often stored or reused as context. If those steps are altered or injected with malicious text, the model's later behavior can be steered, so protecting them directly concerns securing the chain of thought.

Guardrail policies is incorrect. Guardrail policies define what the model is allowed to do, such as blocking unsafe topics. They influence behavior but are not the internal sequence of reasoning steps themselves.

Output formatting is incorrect. Output formatting controls how final answers are structured, like enforcing JSON or templates. It addresses the form of the response, not the underlying reasoning sequence.

Rate limiting is incorrect. Rate limiting restricts how many requests can be made over time. It mitigates abuse volume but does not protect or control the content of intermediate reasoning.

Related Content

resources\questions\q_analyzing_ai_attacks_07.question.xml